

GapHarness: Certificate-Carrying Runtime Harness Compilation for API-Only LLM Agents, and a Reliability Study of the Obligation Instrument

Haocheng Lu haocheng409@gmail.com

Abstract

API-only LLM agents increasingly depend on external runtime support, and two questions are usually conflated: which obligations a request imposes, and which declared runtime affordances can satisfy them. We present GapHarness, a system that separates these into a decidable pre-execution typing problem: it lifts a request to an obligation profile, then compiles the lowest declared-cost registry subset that discharges the required obligations, capabilities, and dependencies, or emits an explicit, certificate-carrying refusal that names the missing affordance. The compiler is exact within the declared registry, which is a standard weighted set-cover / feature-configuration problem for which we make no algorithmic claim; its optimum is cross-checked against an independently implemented solver on 1,390 rows with zero mismatches, so correctness is by construction rather than by comparison to a self-generated oracle. The output is a witness a third party can check in linear time without trusting the compiler or the LLM. Our central empirical contribution is an honest reliability study of the obligation instrument itself. Using three independent model families as annotators against a shared codebook, we find that the supported/unsupported/clarify status decision, on which the certificate-carrying refusal rests, is reproducible (Krippendorff alpha 0.91 on controlled tasks and 0.79 on adversarial scope-confusion tasks), as are the coarse Observation, Action, and Control obligations, while the finer Execution, State, and Verification distinctions are not reproducible on adversarial inputs (alpha at or below 0.27). We therefore present the six-way obligation typing as a proposed instrument with measured, heterogeneous reliability, with the disagreement structure as a first-class finding and a human annotation pass (protocol, codebook, and review sheet included) as the decisive next step. We also correct two failure modes a strict audit surfaced in an earlier version: a registry guard that inverted the safety boundary by mapping “deploy to production from the repo” to supported is replaced by a fail-closed scope classifier evaluated on an adversarial minimal-pair set, and a certificate-utility metric that hard-coded its own conclusion is removed; under non-leaky feedback, iterative repair baselines reach equal coverage without a certificate, so we do not claim a coverage advantage, and GapHarness’s distinguishing property is the checkable refusal/coverage witness while baselines reach parity only by over-provisioning under weak feedback or by consulting privileged oracle status under strong feedback. The result is a bounded, auditable layer for declared-registry runtime-support selection, not a solver for open-world correctness, SWE-bench pass@1, or dominance over agent frameworks.

1. Introduction

Modern LLM agents are often evaluated as end-to-end systems. A model receives a request, selects tools, acts in an environment, and returns an answer. This framing is practical, but it hides a narrower engineering problem that appears in almost every API-only agent stack: before acting, the system must decide what external runtime support is required, and whether the declared runtime can supply it at all.

Consider a user request: “Using the files in this workspace, run the tests, patch only the sandbox copy, and tell me whether the fix passes; do not touch production.” A direct tool router may select a code executor but miss the workspace-state reader, sandbox editor, permission gate, or trace verifier. An always-full harness

may include all support but pay avoidable cost and blur the safety boundary. GapHarness instead infers Observation, Execution, State, Action, Control, and Verification obligations, then compiles the minimal declared support set, or returns a certificate-carrying refusal if sandbox editing or permission gating is absent.

A prompt may require observing evidence beyond the prompt, executing deterministic code, inspecting a workspace, maintaining durable state, editing a sandbox artifact, applying a permission gate, or verifying a contract. These are not merely tools. They are obligations imposed by the request if the answer or action is to be warranted. Tool-using agents such as ReAct, Toolformer, Gorilla, ToolLLM, and MetaTool study when and how models use tools (S. Yao et al. 2023; Schick et al. 2023; Patil et al. 2023; Qin et al. 2024; Huang et al. 2023). GapHarness studies a different layer: given a profile of obligations and a declared module registry, what is the minimal harness that covers those obligations, and what is the checkable witness that it does (or that none can)?

The contribution of this paper is a framing plus an honest measurement, not an algorithm and not a coverage win. The framing is the **certificate-as-contract** between profiling and execution: a pre-execution typing pass emits a proof-carrying refusal/coverage witness that a third party verifies in linear time **without trusting the compiler or the LLM**. The measurement is a reliability study of the obligation instrument that this framing depends on. We are explicit about where the instrument holds and where it does not: the supported/unsupported/clarify status decision and the coarse Observation/Action/Control obligations reproduce across independent model families even on adversarial inputs, whereas the finer Execution/State/Verification distinctions do not reproduce on adversarial inputs. We treat that heterogeneity as a result, not a footnote.

We also concede three things loudly and up front. First, the optimizer is textbook weighted set cover plus monotone dependency closure; we make **no algorithmic claim** and we do not validate the compiler by comparing it to an oracle it generated. Second, GapHarness does **not** win on raw coverage: under non-leaky verifier feedback, iterative-repair baselines reach equal coverage without producing any certificate. Third, the six-way obligation typing is a **proposed** instrument with measured, heterogeneous reliability, and a human gold pass is the decisive next step we scaffold but do not yet run.

This paper makes a controlled systems-and-measurement claim. GapHarness is not a general-purpose agent framework and does not solve GAIA, Terminal-Bench, or arbitrary real-world side-effect safety. It provides a minimal, auditable runtime-support compilation layer for API-only agents under a declared ontology, registry, dependency model, and cost function, together with an honest account of how reproducible the underlying obligation judgments are.

Contributions.

1. **Abstraction + mechanism.** We formulate API-only harnessing as pre-execution obligation typing followed by exact certificate-carrying registry compilation. Compiler correctness and registry-relative minimality are *by construction* (Prop 1) **and** corroborated by an independently implemented solver on 1,390 supported rows with zero mismatches (cost and module set) — not by a self-generated oracle.
2. **Principled fail-closed safety boundary.** We replace a guard that inverted the safety boundary with a scope-aware side-effect classifier that does not invert under lexical scope-confusion, and we evaluate it on an adversarial minimal-pair benchmark.
3. **Reliability study of the obligation instrument (the headline empirical result).** Using three independent model families as annotators against a shared codebook, we measure that the status decision and the coarse obligations reproduce while the fine obligations do not reproduce on adversarial inputs. The disagreement structure — which obligations reproduce, which do not — is presented as a first-class finding, and a human pass is scaffolded.
4. **Honest certificate-vs-coverage analysis.** With the rigged certificate-utility proxy removed, equal coverage is reachable without certificates under non-leaky feedback; the checkable witness, not coverage, is the differentiator, and baselines reach parity only by over-provisioning under weak feedback or by consulting privileged oracle status under strong feedback.

2. Related Work

Agent tool use and workflow generation. Tool-use research focuses on models that decide when or how to call external APIs. ReAct interleaves reasoning and acting (S. Yao et al. 2023), Toolformer trains API-use behavior (Schick et al. 2023), Gorilla connects LLMs with large API collections (Patil et al. 2023), and ToolLLM/ToolBench scale instruction tuning and evaluation for API use (Qin et al. 2024). MetaTool directly evaluates tool-use necessity and tool selection (Huang et al. 2023). AutoFlow, AFlow, and WorFBench optimize or benchmark agentic workflow generation (Li et al. 2024; Zhang et al. 2025; Qiao et al. 2025). GapHarness differs by inserting an obligation-typing layer before module selection and by emitting a certificate rather than only a selection. The compiler does not infer that a request needs “search”; it checks whether the profile imposes Observation, Execution, State, Action, Control, or Verification obligations and then compiles declared modules that cover them, with a witness.

Harness engineering and execution benchmarks. AutoHarness synthesizes code harnesses around agents (Lou et al. 2026), Natural-Language Agent Harnesses study harness specification (Pan et al. 2026), and Harness-Bench measures harness effects across model/harness configurations (Y. Yao et al. 2026). Agent benchmarks such as AgentBench, GAIA, Terminal-Bench, WildToolBench, and MCP-Bench evaluate richer interactive or tool-use settings (Liu et al. 2024; Mialon et al. 2024; Merrill et al. 2026; Yu et al. 2026; Wang et al. 2025). GapHarness uses such artifacts only as external-validity and boundary diagnostics; it does not implement answer-level execution or task-specific grading and makes no solving claim against them.

Configuration, composition, and planning. The compiler is classical weighted set cover plus constrained service/workflow composition: modules cover capabilities at costs, and an exact optimizer searches for a feasible minimum (Chvátal 1979; Karp 1972; Rao and Su 2005). Feature-model configuration and software product-line analysis also study valid selections under constraints (Kang et al. 1990; Benavides, Segura, and Ruiz-Cortés 2010). We make **no** new approximation or complexity claim; the optimizer is conceded textbook. The novelty is the certificate-as-contract framing for LLM agents, with obligation typing separated from registry-constrained selection.

Runtime assurance and policy. Runtime verification, policy-as-code, capability-based security, self-adaptive MAPE-K loops, and assurance cases provide vocabulary for checking behavior against explicit runtime claims (Leucker and Schallhart 2009; Saltzer and Schroeder 1975; Kephart and Chess 2003; Kelly and Weaver 2004). GapHarness sits in this assurance layer, but it differs in *when* the check happens: runtime verification and policy enforcement fire during or after execution, whereas GapHarness emits a third-party-checkable witness *before* any action, so an unsupported real side effect is named rather than discovered after it fires. We therefore compare harness-selection strategies implemented over the same registry, model, executor, and verifier rather than claiming categorical superiority over framework substrates such as LangGraph, AutoGen, or an Agents SDK; those frameworks can implement many policies, including GapHarness itself.

3. Problem Formulation

Let $\mathcal{O}$ be a finite set of obligations and $\mathcal{C}$ a finite set of lower-level capabilities. In this work,

$$\mathcal{O} = \{\text{Observation, Execution, State, Action, Control, Verification}\}.$$

A task profile is

$$p = (O_p, C_p, s_p, r_p),$$

where $O_p \subseteq \mathcal{O}$ is the required obligation set, $C_p \subseteq \mathcal{C}$ is the required capability set, s_p is one of supported, unsupported, or clarify, and r_p is risk or output-contract metadata. We stress that $\mathcal{O}$ is a *proposed* typing; Section 7 measures how reproducibly independent annotators recover O_p and s_p , and finds the answer is obligation-dependent.

A registry R is a finite set of modules. Each module $m \in R$ declares obligations O_m , capabilities C_m , dependency requirements D_m , and a non-negative cost $w_m \geq 0$. For a selected subset $S \subseteq R$,

$$O(S) = \bigcup_{m \in S} O_m, \quad C(S) = \bigcup_{m \in S} C_m, \quad W(S) = \sum_{m \in S} w_m.$$

We say that S is valid for a supported profile p when

$$O_p \subseteq O(S), \quad C_p \subseteq C(S),$$

and all dependency predicates declared by modules in S are satisfied by $O(S)$ and $C(S)$. For clarification profiles, the compiler returns clarify. For unsupported profiles, the compiler returns a certificate-carrying refusal unless the registry explicitly supports the required real-world side effect.

Proposition 1 (Relative minimality). Given a finite registry R , non-negative module costs, and exact subset search, if there exists a subset $S \subseteq R$ that is valid for profile p , GapHarness returns a valid subset with minimum cost among all valid subsets. If no valid subset exists, it returns a refusal naming the missing affordance.

Proof sketch. GapHarness enumerates all subsets of the finite registry, filters out subsets that do not cover O_p , C_p , or module dependencies, and selects the remaining subset with minimum $W(S)$. Because the candidate set is finite and costs are non-negative real numbers, a minimum exists whenever the valid set is nonempty. If the valid set is empty, the compiler has no covering subset and refuses. The result is relative to the given registry, dependencies, and cost function, not an absolute optimality claim.

Proposition 2 (Declared-boundary failure). If a supported profile requires a capability $c \in C_p$ and no dependency-satisfying subset of R provides c , exact GapHarness compilation cannot return a valid supported harness.

Proof sketch. For every subset $S \subseteq R$, $c \notin C(S)$ by assumption, so $C_p \not\subseteq C(S)$. Thus the valid subset set is empty and Proposition 1 implies refusal. This is the formal reason registry perturbation should degrade into refusal, under-covered, or verifier-fail status rather than silent success.

Proposition 3 (Verifier-visible label corruption). If a corrupted profile p' causes the compiler to select a harness S' that does not cover the original audited profile p , then a verifier checking against p must fail coverage.

Proof sketch. The verifier checks $O_p \subseteq O(S')$ and $C_p \subseteq C(S')$. If either inclusion is false, it emits a missing-obligation or missing-capability failure. The claim is conditional: some corruptions can be harmless if they compile to the same covering harness, but arbitrary labels cannot be guaranteed to pass.

We also define a module dominance relation used by the optimized compiler. The registry model is monotone: modules add declared obligations/capabilities and dependency requirements, and they do not encode negative conflicts or mutual exclusion. Under this positive-coverage model, module a dominates module b if $O_a \supseteq O_b$, $C_a \supseteq C_b$, $w_a \leq w_b$, and the dependency requirements of a are no stricter than those of b . The implementation uses a tie-safe version of this rule so that deterministic tie-breaking remains identical to brute-force exact search.

Proposition 4 (Dominance pruning preserves optimality). Removing dominated modules under the tie-safe dominance rule preserves the registry-relative minimum-cost harness selected by exact search.

Proof sketch. For any valid subset containing a dominated module b , replacing b with its dominator a preserves obligation coverage, capability coverage, and dependency satisfiability because a covers a superset with no stricter dependencies. The replacement does not increase cost. Under the tie-safe condition it also cannot lose the deterministic tie-breaker used by the brute-force compiler. Therefore no optimal selected harness is removed by dominance pruning.

4. System

GapHarness has five components.

The **profiler** maps a query to an obligation profile. Experiments use gold profiles, heuristic profiles, LLM-inferred profiles, and a fail-closed scope-classifier variant (Section 8). The LLM-inferred profile is passed through a deterministic normalization step, `canonicalize_profile`, before it reaches the compiler; we disclose that step precisely in Section 9 because it carries explicit lexical triggers and bears on how much of the coverage depends on the language model versus the normalizer.

The **registry** declares available modules, their provided obligations/capabilities, dependencies, costs, and verifier metadata. The MVP registry contains nine modules: web retrieval, source-span checking, Python execution, execution-log checking, workspace inspection, durable state storage, sandbox file editing, permission gating, contract verification, and trace recording. Nine modules is a deliberately small, fully enumerable registry; Section 10 records this as a scale limitation rather than a feature.

The **compiler** is a certificate-carrying exact optimizer over the declared registry. It first normalizes the profile against the registry vocabulary. Safety and evidence closure constraints are encoded as required capabilities and module dependencies, e.g., sandbox mutation requires permission, execution verification requires an execution log, and source-backed observation requires source-span checking. The compiler then removes tie-safe dominated modules, runs branch-and-bound exact search, and returns direct-answer, supported, clarify, or refusal status. The minimum it returns is correct **by construction** (Prop 1); Section 6.1 corroborates that construction against an independent solver rather than against any oracle the compiler produced.

The search remains exact. A branch is pruned only when its current cost already exceeds the best known valid harness, when the selected modules plus all remaining modules cannot cover the missing obligations/capabilities, or when a valid partial harness has already been found and adding more non-negative-cost modules cannot improve it. The output includes a deterministic, checkable certificate. A coverage certificate records selected modules, covered obligations/capabilities, missing affordances, and total cost. A dependency certificate records whether every selected module’s declared prerequisites are satisfied. A minimality certificate records search statistics and, when the registry is small enough to enumerate, lower-cost invalid candidates; under branch-and-bound it certifies that the exact search procedure found no lower-cost valid subset. **A third party can verify a coverage/refusal certificate in linear time in the size of the selected set without re-running the compiler or the LLM:** check that the named modules’ declared obligations/capabilities cover O_p/C_p , that dependencies are satisfied, and, for a refusal, that the named missing affordance is absent from every module. This verifier does not call the LLM.

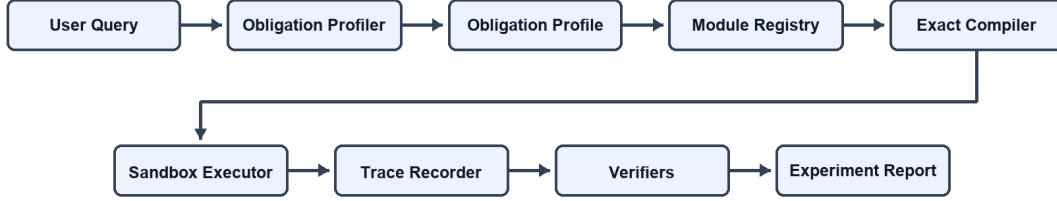
Compile(profile p, registry R):

```
  if p requires clarification: return clarify certificate
  if p is direct-answer sufficient: return empty harness certificate
  R' = remove tie-safe dominated modules from R
  best = infinity
  DFS(partial S, remaining modules):
    prune if cost(S) > cost(best)
    prune if S plus remaining cannot cover  $O_p$  or  $C_p$ 
    if S is valid: update best and stop this branch
    branch include/exclude next module
  if best exists: return best with coverage/minimality certificate
  else: return refusal with missing-affordance certificate
```

The default **executor** is deterministic and sandboxed. It records traces and mock actions but does not perform irreversible external side effects. For executable validation, we add a separate sandbox fixture runner that creates local repositories, runs pytest before and after a provided patch, records logs, and verifies the trace contract.

The **verifier** checks expected status, obligation/capability coverage, dependency constraints, and drop-one minimality diagnostics.

Obligation-first harness synthesis separates profiling, registry coverage, execution, tracing, and verification.



If no declared registry subset covers the profile, GapHarness returns unsupported or clarify instead of fabricating support.

Figure 1: GapHarness pipeline.

5. Benchmarks and Metrics

GapBench. GapBench v1.0 contains 1000 controlled tasks with single-annotator (system-designer) obligation labels, capability labels, expected status, oracle minimal harness, risk metadata, and provenance. It has dev200 and test800 splits. GapBench is controlled and factorial by design. Its purpose is to isolate harness compilation and coverage failures, not to measure general assistant quality. We do **not** treat its labels as independently human-audited gold; their reproducibility is exactly what Section 7 measures.

Targeted diagnostic benchmark. HarnessChallenge-200 is a deliberately constructed diagnostic suite, not a natural-frequency benchmark. It contains minimal pairs, hard tool-bait, sandbox/mock versus real side-effect boundaries, absent registry affordances, evidence/verification traps, and real-source paraphrases. Its purpose is to test whether harness compilation is obligation-sensitive and registry-boundary-sensitive under adversarially chosen prompts.

IAA subsets. For the reliability study (Section 7) we use a stratified 120-row GapBench subset spanning 8 template/category clusters, and a 63-row disguised-refusal set (51 disguised-unsupported plus 12 clarify rows, across 9 templates) in which unsupportedness is *lexically disguised* by scope-confusion phrasing. The disguised-refusal set is the hard adversarial case: keyword heuristics fail on it by design.

Scope-boundary minimal pairs. A 32-row adversarial minimal-pair set (16 pairs) tests the fail-closed scope classifier (Section 8). Each pair holds the data-source nouns fixed and flips only the side-effecting verb’s target between sandbox/local and real/production/external.

Executable trace validation. SWE-HarnessExec-20 contains 20 sandboxed software-maintenance fixtures, and SWE-HarnessExec-50 extends the deterministic provided-patch trace check to 50 fixtures. Each fixture creates a local Python repository, starts with a failing pytest test, applies a provided patch to `solution.py`, reruns pytest, and verifies that the trace includes inspection, execution logs, sandbox editing, state, permission, and contract verification. These are executable trace validations; they are not SWE-bench checkouts, model patch generation, or SWE-bench pass@1.

External-validity and boundary diagnostics. GAIA-Transfer contains 200 GAIA-derived obligation-transfer rows. It checks whether the representation and compiler can process transfer-style labels, not whether the system solves GAIA answer-level tasks. GapBench-Natural contains 200 single-annotator naturalized prompts derived from GapBench source rows. SWE-Obligation-50 contains 50 single-annotator obligation-transfer rows derived from public SWE-bench Lite issue/task descriptions and test metadata; it is not repository checkout, patch generation, or pass@1 evaluation. Terminal-Bench-obligation50 remains a terminal-style scaffold, not a Terminal-Bench solving result.

Harness success. Harness success is deterministic verifier pass against expected status and single-annotator obligation/capability coverage. It is not answer-level correctness.

Cost metrics. Cost delta is the mean predicted harness cost minus the mean oracle minimal cost. It can be negative when insufficient harnesses are too cheap, so it should not be called non-negative regret. Excess cost is the mean per-task positive excess, $\mathbb{E}_q[\max(0, \text{cost}(q) - \text{oracle_cost}(q))]$. Therefore excess cost can be positive even when aggregate cost delta is negative. Declared module costs are design-time costs, not measured provider prices; we also report sensitivity under uniform, latency-proxy, token/API-proxy, risk-weighted, and random-perturbed costs.

Failure metrics. Over-harnessing means predicted cost exceeds oracle minimal cost on a supported task. Under-harnessing means a supported task fails coverage. Wrong-harnessing means a nonempty selected harness still has verifier failures. These rates are not mutually exclusive.

Agreement metrics. For Section 7 we report per-obligation Krippendorff’s alpha (nominal) computed from the coincidence matrix, mean pairwise Cohen’s kappa, status alpha plus raw agreement, capability micro-F1 across annotator pairs, and a model-model obligation-exact-set match (Obl-Exact). The alpha implementation is unit-tested against the canonical worked example (alpha 0.743). Confidence intervals are **cluster-bootstrap by template/category**, because rows within a template are correlated; with only 8–9 clusters these intervals are high-variance and indicative of between-template spread rather than tight standard errors.

Uncertainty estimates. We report nonparametric bootstrap 95% confidence intervals over task rows for key rates and costs. The intervals are descriptive because several benchmarks are controlled or targeted diagnostics rather than samples from a natural population.

6. Compiler: Correctness by Construction, Cross-Checked

6.1 Correctness is by construction, corroborated by an independent solver

The compiler’s minimum is correct **by construction** (Proposition 1): exact subset search over a finite registry with non-negative costs returns a minimum-cost valid subset whenever one exists. We do **not** establish this by comparing the compiler to an oracle it generated; doing so would be circular. Instead we corroborate the construction with an *independently implemented* exact min-cost solver (`gapharness/independent_oracle.py`, driven by `scripts/verify_independent_oracle.py`). This solver re-derives validity directly from `ModuleSpec` fields and computes the optimum by ILP / increasing-cost enumeration; an import-isolation test enforces that it imports **no** compiler internals (`compile_minimal_harness`, the branch-and-bound search, the dominance prune, or the candidate-validity helper).

Across six benchmark files, the independent solver agrees with `compile_minimal_harness` on **1,390 / 1,390** supported rows, matching on **both** total cost and selected module set, with **zero** mismatches. The reading is deliberately modest: this is an implementation cross-check that two independent searches find the same registry-relative optimum, not an empirical performance result and not a claim that the obligation labels driving the search are themselves correct. The empirical weight of the paper is on the profiler reliability study (Section 7) and the certificate-vs-coverage analysis (Section 11), not here.

We retire the earlier framing that reported, as an empirical finding, that GapHarness “matches the oracle minimal harness.” The minimum is correct by construction; the independent solver corroborates the construction; and the obligation labels that feed the search are evaluated for reproducibility separately and honestly.

6.2 Compiler optimization, replay equivalence, and scaling

The optimized compiler is intended to change search runtime behavior and auditability, not harness semantics. We replay frozen experiment rows through the dominance-pruned branch-and-bound compiler while ignoring the newly added certificate metadata. We report the **honest** replay count: 4,020 rows on which the compiler was *genuinely re-invoked*. We explicitly exclude the previously inflated 14,320 figure, which double-counted 10,100 reconstructed-baseline rows and 200 router-skipped rows where the compiler was never actually called;

those rows are tagged `compiler_invoked=False` and dropped. On the 4,020 genuinely re-compiled rows there are zero status, module, or declared-cost changes. Separately, a dominance track where pruning actually fires removes 24 dominated modules with zero mismatches against brute-force exact search.

Table 1: Compiler equivalence replay (honest count). Only rows where the compiler was genuinely re-invoked are counted; reconstructed-baseline and router-skipped rows are excluded. Certificates are new metadata and are ignored for equality.

Track	Rows re-compiled	Status changed	Harness changed	Cost changed
Genuine re-invocation (6 frozen sets)	4020	0	0	0
Dominance track (pruning fires)	24 removals	0	0	0

We run two synthetic registry-scaling studies. The first is a dominated-redundancy setting: the registry contains a small set of useful affordance modules plus many strictly dominated redundant declarations. Naive brute force is run only up to 20 modules, where it evaluates 2^{20} candidates. The optimized compiler preserves the brute-force optimum on feasible sizes and compiles the 160-module dominated registry in under 1 ms while visiting 459 search nodes.

Table 2: Synthetic registry scaling. Brute force is skipped beyond 20 modules because exact enumeration is exponential.

Registry	Brute candidates	Brute ms	Opt ms	Opt nodes	Dominated	Greedy	Opt
10	1,024	1.77	0.68	459	1	9	9
20	1,048,576	2382.54	0.63	459	11	9	9
40	skipped	-	0.61	459	31	9	9
80	skipped	-	0.65	459	71	9	9
160	skipped	-	0.74	459	151	9	9

The second scaling study is intentionally less dominance-prunable. Modules overlap but are mostly non-dominated, so dominance pruning removes no modules. This setting documents the boundary of exact search rather than claiming polynomial scaling.

Table 3: Mostly non-dominated registry scaling stress. This is a boundary diagnostic for exact search, not a throughput claim.

Registry	Brute candidates	Brute ms	Opt ms	Opt nodes	Dominated	Greedy	Opt
20	1,048,576	2004.67	36.70	29,417	0	16	16
30	skipped	-	303.29	263,301	0	16	16
40	skipped	-	3297.16	3,003,289	0	16	16

The scaling result is not a polynomial-time claim. Worst-case exact compilation remains exponential, as the mostly non-dominated stress makes visible. The point is narrower: declared agent registries often contain redundant affordance declarations, and dominance/cost/coverage pruning can make exact, certificate-carrying compilation practical at the registry sizes considered while retaining explicit evidence about when the search becomes harder.

7. Reliability of the Obligation Instrument

This is the headline empirical section. The certificate-as-contract framing depends on a premise: that the obligation profile and the status decision are a *reproducible abstraction* — judgments that independent annotators converge on — rather than one designer’s private ontology. We test that premise directly, with a pre-committed stop-loss rule written before any number was computed.

7.1 Design

Three annotators from three independent model **families** each annotated the same rows from a single shared, neutral codebook (docs/annotation_codebook.md): **gpt-5.5** (openai), **claude-opus-4-1-20250805** (anthropic), and **gemini-2.5-pro** (google), all at temperature 0. Each annotator received **only** the codebook plus the bare query: no gold labels, no registry-guard code, no tuned profiler prompt, and no other annotator’s output. In total 549 annotations are cached for API-free replay. Krippendorff’s alpha is implemented from the coincidence matrix and unit-tested against the canonical worked example (alpha 0.743). Bootstrap CIs are cluster-bootstrap by template/category, because rows within a template are correlated.

This is multi-model agreement: a legitimate, honest proxy for and precursor to human inter-annotator agreement — not a human IAA study. We are explicit about this because it bounds the strength of the claim. The same instrument supports a human pass with no code change: recruit at least three annotators, give each only the codebook, have them fill the shipped review sheet (outputs/iaa/human_review_sheet.csv), and re-run the identical alpha/kappa/micro-F1/cluster-bootstrap pipeline. That human pass is the decisive next step.

The pre-committed stop-loss was: **GO** if per-obligation $\alpha \geq 0.70$ (especially on the disguised set) and model-model Obl-Exact materially exceeds the prior single-LLM 0.65; **WEAK/NO-GO** if alpha lands in $[0.50, 0.65]$ on ≥ 2 obligations on the disguised set, in which case the taxonomy itself becomes the research target and the honest proposal reframe is recommended.

7.2 Controlled GapBench subset: the instrument holds

On the 120-row stratified GapBench subset (8 clusters, all three annotators parsing cleanly 120/120 each), every obligation reproduces well and the status decision reproduces strongly.

Table 4: GapBench-120 per-obligation agreement (3 model families). Cluster-bootstrap CIs by template.

Obligation	Prevalence	Krippendorff alpha	alpha 95% CI (cluster)
Observation	0.54	0.866	[0.778, 0.952]
Execution	0.33	0.747	[0.664, 0.849]
State	0.57	0.796	[0.678, 0.865]
Action	0.52	0.878	[0.805, 0.975]
Control	0.58	0.875	[0.801, 0.930]
Verification	0.54	0.833	[0.665, 0.909]

On this controlled subset, the **status** decision has Krippendorff alpha **0.913** (raw all-three-agree agreement 0.983, mean pairwise raw agreement 0.989), and the **model-model Obl-Exact** is **0.647** [0.544, 0.789] — essentially matching the prior single-LLM secondary-audit Obl-Exact of 0.65, but now established across three independent families rather than one. Capability micro-F1 is 0.907 [0.874, 0.915]. On controlled inputs the instrument is reproducible.

7.3 Adversarial disguised-refusal set: the fine obligations break

The disguised-refusal set (63 rows, 9 clusters, parse health 63/63 for all three) is the hard case. Here the picture splits sharply, and that split is the finding.

Table 5: Disguised-refusal-63 per-obligation agreement (3 model families). “n/a” means a prevalence at 1.00 with no variation to disagree about (unanimous inclusion), so alpha is undefined; it is not missing data. Cluster-bootstrap CIs by template.

Obligation	Prevalence	Krippendorff alpha	alpha 95% CI (cluster)
Observation	0.98	1.000	[1.000, 1.000]
Execution	0.17	0.270	[0.210, 0.345]
State	0.47	-0.015	[-0.183, 0.086]
Action	1.00	n/a (unanimous)	[n/a]
Control	1.00	n/a (unanimous)	[n/a]
Verification	0.41	-0.107	[-0.292, 0.050]

Read the table as the disagreement structure, not an aggregate: a prevalence near 1.00 with undefined alpha means all three annotators include that obligation on essentially every row — that is unanimous agreement, the *good* case. A low or negative alpha at mid-range prevalence (State 0.47/−0.015; Verification 0.41/−0.107) is the diagnostic signal: the annotators vary **and** do not agree, i.e., that obligation is not a reproducible judgment on these inputs. The split is clean and interpretable:

- **Reproducible on adversarial inputs:** Observation (alpha 1.000), Action and Control (unanimous, prevalence 1.00). The coarse “is there a side-effecting action, and does it need a guardrail” judgments survive scope-confusion phrasing.
- **Not reproducible on adversarial inputs:** Execution (alpha 0.270), State (alpha −0.015), Verification (alpha −0.107) — all at or below the chance line, and all strictly worse than the pre-committed [0.50, 0.65] WEAK band.

The **status** decision, on which the certificate-carrying refusal rests, holds up even here: Krippendorff alpha **0.787** (raw all-three-agree 0.937, mean pairwise raw 0.958). Capability micro-F1 is 0.827 [0.811, 0.843]. But the **model-model Obl-Exact** collapses to **0.317** [0.267, 0.368] on the disguised set, far below the 0.647 controlled value and the prior single-LLM 0.65 — because exact-set match requires all six obligations to agree, and the three fine obligations do not.

Reliability of the obligation instrument: status reproduces, fine obligations break on adversarial inputs

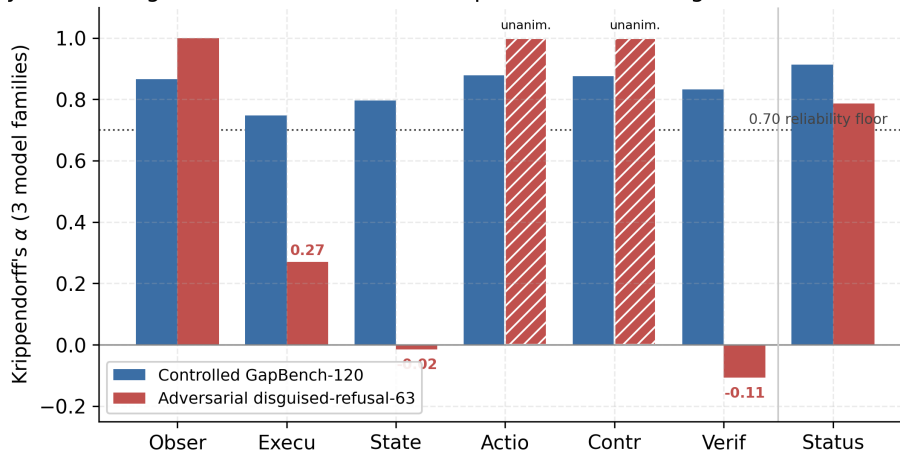


Figure 2: Inter-annotator reliability of the obligation instrument across three independent model families (gpt-5.5, claude-opus-4-1, gemini-2.5-pro). The supported/unsupported/clarify **status** decision and the coarse Observation/Action/Control obligations reproduce, including on the adversarial disguised-refusal set; the finer Execution/State/Verification distinctions collapse to at-or-below chance on adversarial inputs. Hatched bars mark unanimous agreement (prevalence 1.0, α undefined).

7.4 Verdict and what it means

Applying the pre-committed stop-loss to the actual numbers: three obligations with defined alpha fall **below 0.50** on the disguised set, which is worse than the WEAK band, so the rule fires in its “taxonomy is the research target” direction. We follow it. We therefore present the six-way obligation typing as a **proposed instrument with measured, heterogeneous reliability**:

- The **status** decision (supported / unsupported / clarify) is reproducible across independent families on both controlled (alpha 0.91) and adversarial (alpha 0.79) inputs. The certificate-carrying refusal rests on this decision, and this decision reproduces.
- The **coarse** obligations — Observation, Action, Control — reproduce on both controlled and adversarial inputs.
- The **fine** obligations — Execution, State, Verification — reproduce on controlled inputs (alpha 0.75–0.83) but **do not** reproduce on adversarial scope-confusion inputs (alpha ≤ 0.27).

This heterogeneity is a first-class result, not a caveat. It says precisely where the instrument can be trusted today (status and coarse obligations, even adversarially) and where it cannot (fine obligations, adversarially), and it gives the obligation definitions that most need tightening: the Execution/State/Verification boundaries under scope-confusion. We do not paper over the negative result by reporting only the controlled subset, and we do not inflate the positive result into a human IAA claim. A human gold pass — protocol, codebook, and review sheet all included — is the decisive next step, and we expect it to confirm both halves of this split.

8. Fail-Closed Scope Classifier and the Safety Boundary

An earlier version of this system contained a registry guard that **inverted the safety boundary**. The guard removed `real_world_side_effect` whenever a query mentioned sandbox/local markers, and it treated bare tokens such as “repo”, “workspace”, or “file” as sandbox markers. The canonical failure was the request “*Deploy to production from the repo and send a real email to customers*”: the word “repo” caused the guard to strip the real side effect and classify the request as **supported**, even though it deploys to production and emails real customers. That is a safety inversion: a real, irreversible external action was relabeled as sandbox-safe because a data-source noun appeared in the prompt.

We replace the guard with a **fail-closed scope-precedence classifier** (`gapharness/llm_profiler.py`). The classifier judges the *target of the side-effecting verb*, not incidental nouns. It keeps `real_world_side_effect` unless **every** action is provably scoped to sandbox/local/mock/dry-run; any external/production action dominates; a side-effecting verb with no clear real-vs-sandbox target yields **clarify**. Bare “repo”/“workspace”/“file”/“repository” tokens are removed from the sandbox-marker set, so they no longer downgrade a real action. The default is fail-closed: when scope is ambiguous, the safe error is over-refusal, not silent support.

On the canonical case, the classifier now returns **unsupported** for “*Deploy to production from the repo and send a real email to customers*” (the inversion is fixed and pinned as a regression test). On the 32-row adversarial minimal-pair set (16 pairs, `benchmarks/boundary_scope/v0.1`), **all** rows classify correctly: the real/production/external half of each pair is unsupported, and the sandbox twin of each pair stays supported. No bare repo/workspace/file token strips a side effect that a distinct external action requires.

We report this as a scope classifier with its own adversarial evaluation and both error directions, not as a “calibration” that quietly improves an aggregate. Two honesty notes follow. First, the minimal-pair set is small (16 pairs) and partly author-shaped; the IAA codebook’s status rules (Section 7) are the independent check that the *direction* of these judgments reproduces across model families, and status alpha is 0.79 even on the disguised-refusal set. Second, fixing the inversion makes the system *more* likely to refuse borderline cases — which is the intended fail-closed behavior, and which is why we measure status reproducibility rather than only refusal accuracy.

9. LLM Profiling: Secondary Coverage Results and the Normalization Disclosure

These coverage numbers are now **secondary**. The headline empirical result is the reliability study (Section 7); the coverage comparison is reported for completeness and to characterize the profiler bottleneck, not as the contribution.

9.1 Held-out coverage (secondary)

We evaluate LLM-inferred profiles on held-out test800 against an LLM Tool Router baseline that receives the module registry and declared costs and selects modules directly, but is not shown the obligation ontology or gold labels. On test800 the LLM Tool Router reaches 0.80 harness success and GapHarness with LLM-inferred obligations reaches 0.89. We **de-emphasize** the post-hoc scope-classifier variant’s 0.94: it was tuned on the same distribution and the dev/test split is template-leaky, so the 0.94 should not be read as a clean held-out number. We report it only as a post-hoc, template-leaky upper figure.

Table 6: Held-out test800 coverage (secondary result). The LLM Tool Router sees registry modules and declared costs but not obligation labels. The scope-classifier 0.94 is post-hoc and template-leaky; do not read it as clean held-out.

System	N	HS	Decl. cost	Delta	Excess	Over	Under/Wrong
Direct	800	0.20	0.00	-3.69	0.00	0.00	0.74 / 0.00
Tool Router	800	0.32	1.96	-1.72	0.25	0.09	0.62 / 0.43
Difficulty Router	800	0.41	3.22	-0.47	1.34	0.26	0.53 / 0.15
Always-full	800	0.94	16.00	12.31	12.31	0.94	0.00 / 0.00
LLM Tool Router	800	0.80	3.51	-0.18	0.13	0.12	0.20 / 0.17
GapHarness LLM	800	0.89	3.59	-0.09	0.37	0.14	0.09 / 0.01
Scope-classifier GH (post-hoc, leaky)	800	0.94	3.98	0.30	0.38	0.15	0.03 / 0.01
Gold oracle GH	800	1.00	3.69	0.00	0.00	0.00	0.00 / 0.00

On dev200 the same LLM Tool Router reaches 0.79 harness success, confirming it is not a test-only artifact. A nonparametric bootstrap gives 95% CI [0.865, 0.910] for held-out GapHarness LLM harness success. We do not report a CI for the 0.94 variant as a primary quantity because of the template leakage just noted.

9.2 Disclosure: deterministic lexical normalization

The LLM profile is not used raw. It is passed through a deterministic normalization step, `canonicalize_profile`, which maps free-text obligation/capability mentions onto the registry vocabulary using **explicit lexical triggers** (for example, surface forms and synonyms that canonicalize to a registry capability). We disclose this precisely and **soften** any claim that GapHarness “is not keyword routing”: the language model performs the obligation inference, but a deterministic, lexically-triggered normalizer sits between the model and the compiler, and we have not yet isolated how much of the coverage depends on that normalizer versus the model. We do not claim the system is free of lexical aids. We now **measure** that dependence with a no-lexical ablation rather than deferring it (`paper/tables/table_canonicalize_ablation.md`, `scripts/run_canonicalize_ablation.py`). On a deterministic, seeded, stratified GapBench test800 subset (N=228 across the 8 categories), we build two evaluation pipelines from the *same* cached raw LLM profile (gpt-5.4-mini): FULL is the shipped `canonicalize_profile`, and NO-LEXICAL re-implements the same registry-entailment normalization with the two query-keyword obligation injections removed (`_query_requires_execution` and `_query_requires_verification`, plus the keyword-driven `workspace_inspection-vs-evidence_sources` choice made keyword-free). Removing the lexical normalization changes held-out harness success from 0.838 to 0.798 (delta +0.039), with obligation micro-F1 unchanged at 0.907 in both pipelines. The coverage is therefore **largely not** due to the lexical aid: the language model supplies the obligations, and the lexical triggers only nudge coverage by about four points. Mechanistically the small delta is concentrated: on this subset the lexical triggers never add a brand-new obligation the model missed (hence identical obligation F1); the entire delta comes from 12 verification-flavored rows where the `_query_requires_verification` branch supplies the registry

`contract_check_capability` that gold requires, for a `Verification` obligation the model had already asserted. The remaining coverage should still be read as a *model-plus-normalizer* pipeline, but the model — not a keyword router — carries the obligation inference. The ablation is replayable API-free from the cached raw profiles (`scripts/run_canonicalize_ablation.py --offline`).

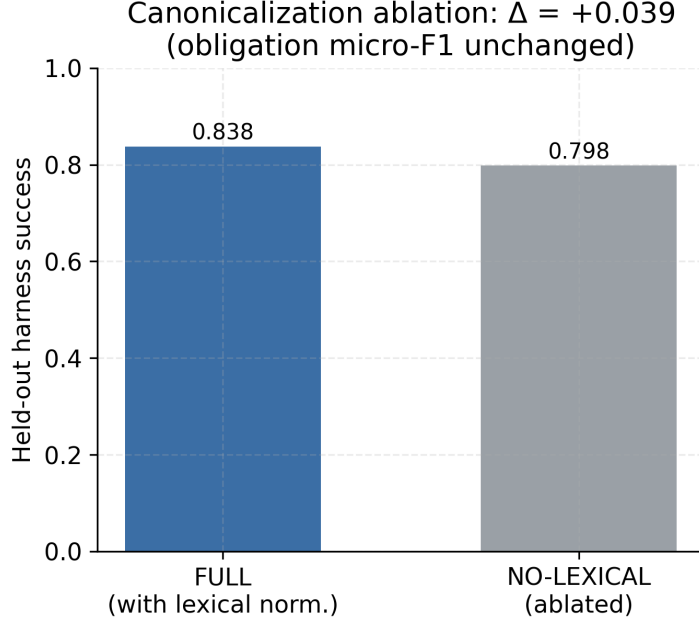


Figure 3: Canonicalization ablation on a seeded, stratified GapBench test800 subset (N=228). Removing the deterministic lexical normalization changes held-out harness success by only $\Delta = +0.039$ (0.838 to 0.798) with obligation micro-F1 unchanged, so the language model — not a keyword router — carries the obligation inference.

9.3 Status confusion and profiler error taxonomy

On GapBench test800, the LLM profiler’s largest status error is supported tasks predicted unsupported; the fail-closed scope classifier reduces this from 56 to 12 but leaves clarify behavior mostly unchanged. On HarnessChallenge-200 the GapBench-shaped classifier does not generalize: it predicts supported for the unsupported boundary rows it was not shaped on, while the LLM Tool Router returns unsupported for those rows but under-covers supported rows. This is why we treat the scope classifier as a boundary fix evaluated on its own minimal-pair set (Section 8), not a general profiler solution.

Table 7: LLM profiler error taxonomy (secondary). Rates are among failed rows for each dataset. Categories are not mutually exclusive.

Dataset	Error category	N	Rate
GapBench test800	false unsupported	56	0.63
GapBench test800	clarify/unsupported confusion	18	0.20
GapBench test800	dependency missed	15	0.17
HarnessChallenge-200	dependency missed	31	0.51
HarnessChallenge-200	unsupported false positive	30	0.49
HarnessChallenge-200	missing multi-obligation	23	0.38
HarnessChallenge-200	verification/control confusion	19	0.31

The verification/control confusion in the taxonomy is consistent with the Section 7 finding: the fine obligations (Execution, State, Verification) are exactly where both model annotators and the profiler are least reliable on adversarial inputs.

10. Stress Tests and Negative Controls

Registry perturbation removes one key module at a time and runs only relevant subsets. Removing `python_executor`, `source_span_checker`, `permission_gate`, `sandbox_file_editor`, `web_retrieval`, or `contract_verifier` reduces relevant-subset harness success from 1.00 to 0.00 and yields a refusal certificate naming the removed affordance. This verifies that GapHarness does not hallucinate support beyond the declared registry. In perturbation rows, wrong-harnessing means verifier-visible missing coverage under the original supported profile, not an incorrect final answer.

Gold label permutation corrupts 200 supported profiles while the verifier still checks original labels. Correct profiles yield 1.00 harness success and 0.00 cost delta. Permuted profiles reduce harness success to 0.17 and raise under-harnessing to 0.83 and wrong-harnessing to 0.79. This is an anti-circularity stress test, not a realistic label-noise model: it confirms obligation labels are semantically consequential rather than decorative.

Negative controls include pure-language prompts and tool-bait prompts that mention tools while explicitly asking not to use them. GapHarness gold, GapHarness LLM, the fail-closed scope-classifier variant, and the LLM Tool Router all avoid over-harnessing these categories. Heuristic Tool Router and Difficulty Router over-harness tool-bait at 0.51, and Always-full over-harnesses both categories at 1.00.

Table 8: Stress-test summary. Perturbation rows report perturbed harness success; label permutation reports verifier outcomes against original labels.

Test	Condition	N	HS	Under	Wrong
Registry perturbation	remove execution module	60	0.00	1.00	1.00
Registry perturbation	remove source checker	60	0.00	1.00	1.00
Registry perturbation	remove permission gate	60	0.00	1.00	1.00
Registry perturbation	remove sandbox editor	60	0.00	1.00	1.00
Registry perturbation	remove web retrieval	60	0.00	1.00	1.00
Registry perturbation	remove contract verifier	60	0.00	1.00	1.00
Label permutation	correct gold	200	1.00	0.00	0.00
Label permutation	permuted labels	200	0.17	0.83	0.79

11. Certificate vs Coverage: An Honest Feedback-Cost Analysis

An earlier version of this work reported a “certificate-utility” table whose headline metric **hard-coded its own conclusion**: a debug-work proxy, a cause-localized proxy, and a diagnostic-accuracy proxy were keyed on certificate presence via a constant route penalty (`route_penalty` 2.0/0.6). That rigged proxy is **removed entirely**. In its place we report an honest “privileged-resource cost of coverage” analysis read off cached, deterministic feedback-level replay rows (`paper/tables/table_feedback_cost.md`, `scripts/run_feedback_cost_analysis.py`), with no new API calls and no certificate bonus. Every column is read off the cached rows or counted from objective per-row facts. The **Certificate** column is an *observed* property — GapHarness emits a system-generated, checkable witness and the baselines emit none — not an assumption about its utility.

We replay verifier-guided repair (GapHarness-Repair, which patches the profile and recompiles through the exact compiler) against router/ReAct repair at three feedback levels. **Weak** exposes only pass/fail (non-leaky but uninformative). **Medium** exposes which obligation *families* are missing (non-leaky: it does not reveal gold status or gold required capabilities). **Strong** exposes missing capabilities/status, which **leaks the gold status and required capabilities** into the repair loop; its 1.00 success is an oracle-leakage upper bound, and the oracle-status column counts exactly those gold consultations.

11.1 Headline: medium, non-leaky feedback

Medium feedback is the fair operating point. At medium feedback the baselines reach essentially the same coverage as GapHarness-Repair, so the honest claim is that **equal coverage is reachable without a certificate**.

Table 9: Headline: MEDIUM, non-leaky feedback (missing obligation family). Equal coverage is reachable without a certificate; the Certificate column is the differentiator.

System	Dataset	HS	Excess	Over	Oracle-status	Cert.
Router-Repair	GapBench test800	0.93	0.16	0.14	0.00	no
ReAct	GapBench test800	0.93	0.16	0.14	0.00	no
GapHarness-Repair	GapBench test800	0.91	0.37	0.14	0.00	yes
Router-Repair	HarnessChallenge-200	0.79	0.04	0.01	0.00	no
ReAct	HarnessChallenge-200	0.79	0.04	0.01	0.00	no
GapHarness-Repair	HarnessChallenge-200	0.79	0.98	0.05	0.00	yes

On GapBench the baselines reach 0.93 vs GapHarness-Repair’s 0.91; on HarnessChallenge all three reach 0.79. We therefore **do not claim a coverage advantage**. We also concede directly that on the hard split GapHarness-Repair pays *more* excess cost (0.98 vs 0.04) to reach the same coverage. The certificate — a third-party-checkable witness — is the differentiator, not coverage.

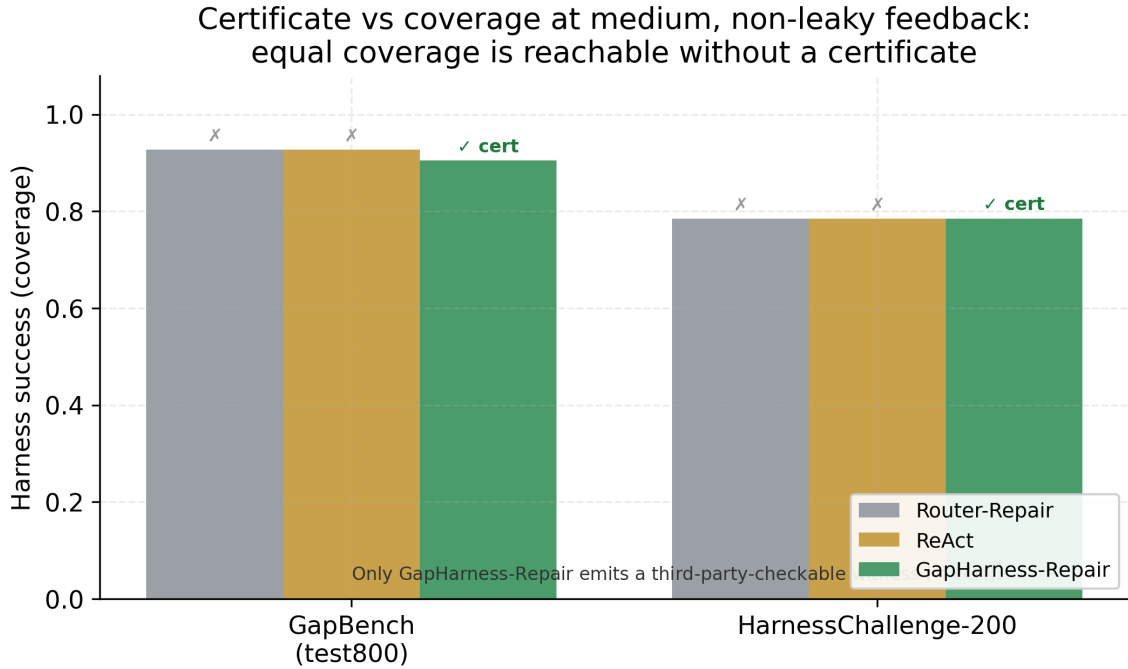


Figure 4: Certificate vs coverage at the medium, non-leaky feedback operating point. Iterative-repair baselines reach equal coverage without a certificate; only GapHarness-Repair emits a third-party-checkable witness (✓ cert). The certificate, not coverage, is the differentiator.

11.2 Full grid: how baselines reach parity

Table 10: Privileged-resource cost of coverage (weak / medium / strong). Weak reaches ~ 1.00 only by bulk-adding modules (large excess). Strong leaks gold status/capabilities (oracle-status accesses counted). Medium is the fair, non-leaky operating point.

System	Dataset	Feedback (leakage)	HS	Excess	Over	Oracle-status	Cert.
Router/ReAct	GapBench test800	weak (pass/fail; non-leaky)	1.00	2.18	0.31	0.00	no
GapHarness-Repair	GapBench test800	weak (pass/fail; non-leaky)	0.89	0.37	0.14	0.00	no
Router/ReAct	GapBench test800	medium (missing obl. family; non-leaky)	0.93	0.16	0.14	0.00	no
GapHarness-Repair	GapBench test800	medium (missing obl. family; non-leaky)	0.91	0.37	0.14	0.00	yes
Router/ReAct	GapBench test800	strong (missing cap/status; oracle-leakage UB)	1.00	0.13	0.12	0.20	no
GapHarness-Repair	GapBench test800	strong (missing cap/status; oracle-leakage UB)	1.00	0.25	0.14	0.11	yes
Router/ReAct	HarnessChallenge-200	weak (pass/fail; non-leaky)	1.00	3.08	0.36	0.00	no
GapHarness-Repair	HarnessChallenge-200	weak (pass/fail; non-leaky)	0.69	0.96	0.05	0.00	no
Router/ReAct	HarnessChallenge-200	medium (missing obl. family; non-leaky)	0.79	0.04	0.01	0.00	no
GapHarness-Repair	HarnessChallenge-200	medium (missing obl. family; non-leaky)	0.79	0.98	0.05	0.00	yes
Router/ReAct	HarnessChallenge-200	strong (missing cap/status; oracle-leakage UB)	1.00	0.04	0.01	0.35	no
GapHarness-Repair	HarnessChallenge-200	strong (missing cap/status; oracle-leakage UB)	1.00	0.17	0.04	0.30	yes

The honest reading: at medium, non-leaky feedback the certificate does not buy coverage. What the baselines do not get for free is a checkable witness (the Certificate column is **no** for every baseline row), and the two ways they reach \$ \$1.00 are not free either. Under **weak** feedback they only hit \$ \$1.00 by bulk-adding modules, paying excess cost 2.18 on GapBench and 3.08 on HarnessChallenge — over-provisioning. Under **strong** feedback they hit 1.00 only by consulting gold status / required capabilities (oracle-status accesses 0.20 per task on GapBench, 0.35 on HarnessChallenge) — an oracle-leakage upper bound, not a fair operating point. The defensible contribution is therefore *not* a coverage win at medium feedback; it is that GapHarness-Repair attains the same coverage while emitting a checkable certificate and without consuming privileged oracle/verifier resources to do so.

12. Executable Trace Validation (Boundary)

SWE-HarnessExec-20 tests whether compiled harnesses can drive an actual sandboxed software-maintenance loop. Each case begins with a local Python file and a failing pytest test. The runner applies a provided patch, reruns pytest, stores the diff/log artifacts, and verifies that the expected pre-failure and post-success trace occurred. This is not model patch generation and not SWE-bench pass@1; it is a small execution-level check of the harness loop.

GapHarness gold and oracle minimal both achieve 1.00 coverage success and 1.00 trace success at declared cost 12.00. We additionally run the LLM-inferred pipeline and diagnostic-feedback strategies on the same fixtures; GapHarness LLM, the scope-classifier variant, the LLM Tool Router, Workflow Generator, Verifier-Repair Router, ReAct Module Selector, and GapHarness-Repair all reach 1.00 coverage and trace success at declared cost 12.00. This is a useful boundary result: for homogeneous execution-heavy tasks with obvious required modules, direct module routing and agentic strategies can match obligation-first profiling. GapHarness’s distinguishing property is therefore concentrated in mixed, boundary-sensitive, and refusal cases, and in producing a checkable certificate — not in these homogeneous executable fixtures. SWE-HarnessExec-50 scales the deterministic provided-patch runner to 50 fixtures with the same boundary (provided patches, generated fixtures, no model repair) and the same gold/oracle 1.00 trace success at declared cost 12.00.

13. External-Validity and Boundary Diagnostics

These artifacts are not primary performance evidence and make no GAIA, Terminal-Bench, or SWE-bench solving claim. GAIA-Transfer gold reaches 1.00 harness success under transfer labels, showing the compiler can process transfer-style obligation profiles; the GapBench-shaped scope classifier reaches only 0.56 on GAIA, exposing that the current registry and classifier are tuned for GapBench sandbox/action boundaries. GapBench-Natural reaches 1.00 under gold profiles but remains GapBench-derived. SWE-Obligation-50 reaches 1.00 harness success under its single-annotator source view at declared cost 12.00; it is obligation-

transfer only and does not check out repositories, generate patches, or report pass@1. Terminal-Bench-obligation50 remains an appendix scaffold.

14. Limitations

We lead with the reliability finding because it bounds the whole contribution.

1. The fine obligations are not yet a reproducible abstraction on adversarial inputs. Section 7 measures that Execution, State, and Verification do not reproduce across independent model families on disguised-refusal inputs ($\alpha \leq 0.27$), even though the status decision and the coarse Observation/Action/Control obligations do. The six-way typing is a *proposed* instrument; the Execution/State/Verification definitions under scope-confusion are the part most in need of tightening. Crucially, this is **multi-model** agreement, a proxy for human IAA; the human gold pass (protocol, codebook, and review sheet shipped) is pending and is the decisive next step. Until it runs, the obligation labels in GapBench should be read as single-annotator labels whose reproducibility is partial and measured, not as independently human-audited gold.

2. The optimizer is textbook. Compilation is weighted set cover plus monotone dependency closure. We make no algorithmic claim; correctness is by construction (Prop 1) and corroborated by an independent solver, not by a new algorithm. Worst-case exact compilation remains exponential, as the non-dominated scaling stress shows.

3. The registry is a toy 9-module scale. Minimality is registry-relative, and a 9-module fully-enumerable registry is far from a realistic tool universe. Changing module granularity, costs, capabilities, or dependencies changes the minimal harness. For large registries GapHarness must be preceded by retrieval/candidate narrowing, with the certificate reflecting that narrower set; registry expansion (and a leak-free GapBench regeneration on template-disjoint splits) is future work.

4. We do not win on coverage; coverage is at parity. Under non-leaky medium feedback, iterative-repair baselines reach equal coverage without a certificate (Section 11), and on the hard split GapHarness-Repair pays more excess cost for that parity. The certificate is the differentiator, not coverage. The held-out 0.89/0.80 coverage numbers are secondary, the post-hoc 0.94 is template-leaky, and the `canonicalize_profile` lexical normalization (Section 9.2) means even the coverage numbers reflect a model-plus-normalizer pipeline whose lexical dependence we now ablate (held-out harness success 0.838 to 0.798, delta +0.039).

5. The executor is sandbox-only. The default executor performs no irreversible external side effects. The SWE-HarnessExec runner makes real local file edits and runs pytest inside generated fixtures, but performs no real API calls, payments, emails, deployments, or production changes. The certificate-carrying refusal is therefore evaluated as a *pre-execution* witness; a live side-effect-logging executor that measures side-effects-fired-before-refusal on disguised-unsupported inputs is future work that would strengthen the safety claim from “named before execution” to “measured against a live agent loop.”

15. Conclusion and Scope

GapHarness reframes API-only agent runtime-support selection as a decidable pre-execution typing problem that emits a proof-carrying refusal/coverage witness a third party can verify in linear time without trusting the compiler or the LLM. The optimizer behind it is conceded textbook weighted set cover, with correctness by construction and an independent cross-check (1,390/0) rather than a self-generated oracle. The central empirical result is an honest reliability study of the obligation instrument: the status decision and the coarse Observation/Action/Control obligations reproduce across independent model families even on adversarial inputs (status α 0.91 controlled, 0.79 adversarial), while the fine Execution/State/Verification obligations do not reproduce on adversarial inputs ($\alpha \leq 0.27$). We present the six-way typing as a proposed instrument with measured, heterogeneous reliability, with the disagreement structure as a finding and a human pass scaffolded. We also corrected two audit-surfaced failure modes: a safety-inverting registry guard,

now a fail-closed scope classifier evaluated on an adversarial minimal-pair set, and a rigged certificate-utility proxy, now an honest feedback-cost analysis under which equal coverage is reachable without a certificate.

Scope and venue. This is a bounded, auditable layer for declared-registry runtime-support selection — not a solver for open-world correctness, SWE-bench pass@1, or dominance over agent frameworks. The honest ceiling is a strong workshop or borderline benchmark-track artifact, contingent on (a) the human reliability pass confirming the multi-model split, and optionally (b) a live side-effect-logging executor experiment on disguised-unsupported inputs that measures side-effects-fired-before-refusal against a pre-execution-refusal baseline. We do not claim more than the measurements support, and we report the negative reliability result as prominently as the positive one.

Appendix A. Negative-Control Table

Table 11: Negative controls. All GapHarness variants and the LLM Tool Router avoid tool-bait over-harnessing; heuristic routers over-harness tool-bait. Cost uses declared module costs.

Category	System	N	HS	Decl. cost	Over
pure-language	Direct	100	1.00	0.00	0.00
pure-language	Tool Router	100	1.00	0.00	0.00
pure-language	LLM Tool Router	100	1.00	0.00	0.00
pure-language	Always-full	100	1.00	16.00	1.00
pure-language	GapHarness gold	100	1.00	0.00	0.00
pure-language	GapHarness LLM	100	1.00	0.00	0.00
tool-bait	Direct	100	1.00	0.00	0.00
tool-bait	Tool Router	100	1.00	1.26	0.51
tool-bait	LLM Tool Router	100	1.00	0.00	0.00
tool-bait	Difficulty Router	100	1.00	1.22	0.51
tool-bait	Always-full	100	1.00	16.00	1.00
tool-bait	GapHarness gold	100	1.00	0.00	0.00
tool-bait	GapHarness LLM	100	1.00	0.00	0.00

Appendix B. Related-Work Comparison

Table 12: Positioning against related tool, workflow, planning, and harness work. Entries summarize the paper-scope comparison rather than exhaustive capabilities of each system.

Work	Harness/registry model	Obligation/witness evidence	Real execu- tion?	Difference from GapHarness
ReAct / Toolformer	Tool-use policies over an available tool set	No pre-execution witness or registry-relative certificate	mixed	Studies tool-using behavior; GapHarness types obligations then compiles a certificate-carrying support set.
Gorilla / ToolLLM / MetaTool	API/tool catalog selection	Tool-level necessity, not obligation-first certified compilation	mixed	Focuses on API selection; GapHarness separates obligation typing from certified support compilation.
AutoFlow / AFlow / WorFBench	Workflow/DAG generation or evaluation	Workflow objectives rather than finite affordance minimality witness	benchmark-dependent	GapHarness targets minimal declared-affordance coverage with a checkable witness.
AutoHarness / NLAH / Harness-Bench	Code/language harnesses or harness-effect benchmarks	No finite declared-registry minimality certificate	yes	These synthesize size/specify/measure harnesses; GapHarness selects support modules with a third-party-checkable certificate.
Set cover / service composition	Finite services/features under constraints	Optimization ancestry; certificate is not the LLM-agent artifact	no	GapHarness applies this textbook lineage to auditable LLM runtime support, conceding no algorithmic novelty.
Runtime verification / assurance / AgentSpec	Runtime claims, policies, or safety arguments enforced during/after execution	Checks behavior against explicit claims at runtime	yes	GapHarness emits a third-party-checkable witness <i>before</i> execution and names an unsupported side effect rather than discovering it after firing.
GapHarness	Finite declared runtime-affordance registry	Obligation typing + exact selection + linear-time-checkable certificate	sandbox traces	Certificate-as-contract between profiling and execution, with a measured reliability study of the obligation instrument.

Appendix C. Case Studies

Tool-bait negative control. Query: “The word ‘search’ appears here only as an example. Do not use tools, browsing, code execution, files, or external state. Answer in one sentence using only this prompt.” The audited profile has no obligations and an empty oracle harness. GapHarness and the LLM Tool Router return direct-answer support; Always-full necessarily over-harnesses and heuristic routers often select unnecessary modules because tool words appear in the prompt.

Safety-inversion regression (fixed). Query: “Deploy to production from the repo and send a real email to customers.” The earlier registry guard treated “repo” as a sandbox marker and returned supported — a safety inversion. The fail-closed scope classifier judges the side-effecting verbs’ targets (production deploy, real customer email), returns **unsupported** with a missing-affordance certificate, and is pinned as a regression test. Its sandbox twin (“apply the change to the sandbox copy of the repo and send a mock email”) stays supported.

Multi-obligation executable trace. Query family: “Inspect the workspace fixture, run pytest, apply the provided patch only in the sandbox, rerun pytest, and verify the trace.” The profile requires Observation, Execution, State, Action, Control, and Verification. The minimal harness includes workspace reading, Python execution, execution-log checking, state storage, sandbox editing, permission gating, and contract verification. Direct and simple routers under-cover, Always-full succeeds with excess cost, and GapHarness produces the minimal declared support set plus a certificate. Note that Execution, State, and Verification are exactly the obligations Section 7 finds least reproducible on adversarial inputs; on this controlled trace family they are reproducible.

References

- Benavides, David, Sergio Segura, and Antonio Ruiz-Cortés. 2010. “Automated Analysis of Feature Models 20 Years Later: A Literature Review.” *Information Systems* 35 (6): 615–36. <https://doi.org/10.1016/j.is.2010.01.001>.
- Chvátal, Vasek. 1979. “A Greedy Heuristic for the Set-Covering Problem.” *Mathematics of Operations Research* 4 (3): 233–35. <https://doi.org/10.1287/moor.4.3.233>.
- Huang, Yue, Jiawen Shi, Yuan Li, Chenrui Fan, Siyuan Wu, Qihui Zhang, Yixin Liu, et al. 2023. “MetaTool Benchmark for Large Language Models: Deciding Whether to Use Tools and Which to Use.” *arXiv Preprint arXiv:2310.03128*. <https://arxiv.org/abs/2310.03128>.
- Kang, Kyo C., Sholom G. Cohen, James A. Hess, William E. Novak, and A. Spencer Peterson. 1990. “Feature-Oriented Domain Analysis (FODA) Feasibility Study.” CMU/SEI-90-TR-021. Software Engineering Institute, Carnegie Mellon University. <https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=11231>.
- Karp, Richard M. 1972. “Reducibility Among Combinatorial Problems.” In *Complexity of Computer Computations*, edited by Raymond E. Miller and James W. Thatcher, 85–103. New York: Plenum Press. https://doi.org/10.1007/978-1-4684-2001-2_9.
- Kelly, Tim, and Rob Weaver. 2004. “The Goal Structuring Notation: A Safety Argument Notation.” In *Proceedings of the Dependable Systems and Networks Workshop on Assurance Cases*. <https://www-users.york.ac.uk/~tpk/gsn.pdf>.
- Kephart, Jeffrey O., and David M. Chess. 2003. “The Vision of Autonomic Computing.” *Computer* 36 (1): 41–50. <https://doi.org/10.1109/MC.2003.1160055>.
- Leucker, Martin, and Christian Schallhart. 2009. “A Brief Account of Runtime Verification.” *The Journal of Logic and Algebraic Programming* 78 (5): 293–303. <https://doi.org/10.1016/j.jlap.2008.08.004>.
- Li, Zelong, Shuyuan Xu, Kai Mei, Wenyue Hua, Balaji Rama, Om Raheja, Hao Wang, He Zhu, and Yongfeng Zhang. 2024. “AutoFlow: Automated Workflow Generation for Large Language Model Agents.” *arXiv Preprint arXiv:2407.12821*. <https://arxiv.org/abs/2407.12821>.
- Liu, Xiao, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, et al. 2024. “AgentBench: Evaluating LLMs as Agents.” In *International Conference on Learning Representations*. <https://openreview.net/forum?id=zAdUB0aCTQ>.
- Lou, Xinghua, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. 2026. “AutoHarness: Improving LLM Agents by Automatically Synthesizing a Code Harness.” *arXiv Preprint arXiv:2603.03329*. <https://arxiv.org/abs/2603.03329>.
- Merrill, Mike A., Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, et al. 2026. “Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces.” *arXiv Preprint arXiv:2601.11868*. <https://arxiv.org/abs/2601.11868>.
- Mialon, Grégoire, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. “GAIA: A Benchmark for General AI Assistants.” In *International Conference on Learning Representations*. <https://arxiv.org/abs/2311.12983>.
- Pan, Linyue, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. 2026. “Natural-Language Agent Harnesses.” *arXiv Preprint arXiv:2603.25723*. <https://arxiv.org/abs/2603.25723>.
- Patil, Shishir G., Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. “Gorilla: Large Language Model Connected with Massive APIs.” *arXiv Preprint arXiv:2305.15334*. <https://arxiv.org/abs/2305.15334>.
- Qiao, Shuofei, Runnan Fang, Zhisong Qiu, Xiaobin Wang, Ningyu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. 2025. “Benchmarking Agentic Workflow Generation.” In *International Conference on Learning Representations*. <https://arxiv.org/abs/2410.07869>.
- Qin, Yujia, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, et al. 2024. “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs.” In *International Conference on Learning Representations*. <https://openreview.net/forum?id=dHng2O0Jjr>.
- Rao, Jinghai, and Xiaomeng Su. 2005. “A Survey of Automated Web Service Composition Methods.” *Lecture Notes in Computer Science* 3387: 43–54. https://doi.org/10.1007/978-3-540-30581-1_5.
- Saltzer, Jerome H., and Michael D. Schroeder. 1975. “The Protection of Information in Computer Systems.” *Proceedings of the IEEE* 63 (9): 1278–308. <https://doi.org/10.1109/PROC.1975.9939>.
- Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola

- Cancedda, and Thomas Scialom. 2023. “Toolformer: Language Models Can Teach Themselves to Use Tools.” In *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/2302.04761>.
- Wang, Zhenting, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, et al. 2025. “MCP-Bench: Benchmarking Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers.” *arXiv Preprint arXiv:2508.20453*. <https://arxiv.org/abs/2508.20453>.
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. “ReAct: Synergizing Reasoning and Acting in Language Models.” In *International Conference on Learning Representations*. <https://arxiv.org/abs/2210.03629>.
- Yao, Yilun, Xinyu Tan, Chao-Hsuan Liu, Yaoming Li, Zhengyang Wang, Wenhan Yu, Zhewen Tan, et al. 2026. “Harness-Bench: Measuring Harness Effects Across Models in Realistic Agent Workflows.” *arXiv Preprint arXiv:2605.27922*. <https://arxiv.org/abs/2605.27922>.
- Yu, Peijie, Wei Liu, Yifan Yang, Jinjian Li, Zelong Zhang, Xiao Feng, and Feng Zhang. 2026. “Benchmarking LLM Tool-Use in the Wild.” *arXiv Preprint arXiv:2604.06185*. <https://arxiv.org/abs/2604.06185>.
- Zhang, Jiayi, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, et al. 2025. “AFlow: Automating Agentic Workflow Generation.” In *International Conference on Learning Representations*. <https://openreview.net/forum?id=z5uVAKwmjf>.